# 15.776 Intensive Hands-On Deep Learning

## Final Project: Debunking Fake Disaster Announcements

Giuseppe Iannone (peppegius@mit.edu), Luca Sfragara (lsfragar@mit.edu),
Trisha Sutivong (trishsu@mit.edu), Hanna Zhang (zeyuzh@mit.edu)
Link to Colab

# 1 Introduction

Social media platforms, particularly Twitter (now X), emerge as critical communication channels during emergencies due to their rapid information dissemination and widespread mobile accessibility. Emergency response agencies, news organizations, and policymakers increasingly rely on Twitter to monitor real-time developments and coordinate disaster relief efforts. However, the platform's openness creates a significant challenge: distinguishing real disaster reports from misinformation in time-sensitive scenarios requires automation capable of classifying high volumes of content rapidly.

The technical complexity of this task stems from the inherent characteristics of tweets as data. Twitter content is unstructured, exhibits substantial variation in tone, structure, and length, and lacks systematic linguistic markers that reliably distinguish authentic from fabricated disaster announcements. The nuance of individual tweets depends heavily on contextual cues and patterns that typically require human intuition to interpret. Consequently, the effectiveness of any automated classification approach hinges on its ability to capture and leverage these subtle linguistic and contextual features that make up human language.

# 2 Data

## 2.1 Data Overview

The dataset originates from the Kaggle competition *Natural Language Processing with Disaster Tweets* and comprises 7,613 labeled tweets for training and 3,263 unlabeled tweets for evaluation. Each instance contains:

- **Text:** The tweet content with a maximum length of 280 characters
- **Keyword:** Optional disaster-related keyword (e.g., "wildfire", "earthquake")
- **Location:** Optional user-provided location information
- **Target:** Binary label indicating disaster authenticity (1 = real disaster, 0 = not a disaster)

**Data Splitting:** We partition the labeled training data into three stratified subsets: 68% for training, 17% for validation, and 15% for testing. The stratification ensures proportional representation of disaster and non-disaster tweets across all splits. We first separate the 15% test set, then further divide the remaining data into training and validation sets using a secondary 80-20 split. The Kaggle-provided unlabeled submission set remains unused in our analysis.

**Data Source and Acknowledgment:** As this dataset derives from a publicly accessible Kaggle competition with prior submissions available online, we acknowledge upfront that we may reference these resources for inspiration and methodological guidance. However, the modeling approach, implementation, and analysis presented in this work represent our original contributions derived from class and recitation materials.

## 2.2 Supplemental Data

Beyond the raw tweet text, we engineer a set of numerical features designed to capture structural and stylistic characteristics that may correlate with disaster tweet authenticity. The objective here is to enrich the dataset with numerical features, allowing us to also perform quantitative analysis to examine correlation between

characteristics of a tweet and our target value. We hope to capture distinct linguistic patterns exhibited by disaster tweets that manifest in message length, urgency markers, and social media conventions.

We extract seven numerical features from each tweet:

1. **Character Count:** total number of characters in the tweet
2. **Word Count:** total number of space-separated tokens
3. **Average Word Length:** mean character length per word
4. **Hashtag Count:** number of hashtags, reflecting social media engagement patterns
5. **Mention Count:** number of user mentions (words beginning with @), i.e., direct communication
6. **URL Count:** number of embedded links, which may signal news sharing or source citation
7. **Uppercase Word Count:** number of fully capitalized words (excluding single characters)

These features are computed identically across training, validation, and test sets to ensure consistency. We then treat this set of features as another "knob" we can turn on and off in the following ablation study.

# 3 Methods

We develop and evaluate a progression of NLP models, ranging from classical bag-of-words approaches to modern transformer architectures. Our modeling pipeline consists of five sequential steps designed to explore the impact of different representation strategies and feature engineering techniques.

- **Baseline Model:** We establish a naive baseline by predicting the majority class (non-disaster tweets) for all instances.
- **Bag-of-Words Models:** We implement three variants of bag-of-words (BoW) models:
  - *Model 1: Basic BoW.* A simple feed-forward neural network with no hyperparameter tuning
  - *Model 2: BoW with Hyperparameter Tuning.* A feed-forward neural network with variations in the number of hidden layers, neurons per layer, dropout rates, and learning rates.
  - *Model 3: BoW with Feature Engineering.* A feed-forward neural network with the seven engineered numerical features described in Section 2.2.
- **Transformer-Based Models:** To capture contextual relationships, we implement transformer encoder architectures:
  - *Model 4: Custom Transformer Encoder.* A transformer encoder with multi-head self-attention with just textual data
  - *Model 5: Transformer with Feature Engineering.* A transformer encoder with multi-head self-attention with both engineered features and textual data.
- **Transfer Learning: BERT Models.** We fine-tune two pre-trained models from the Hugging Face library:
  - *Model 6: RoBERTa.* A robustly optimized BERT pretraining approach.
  - *Model 7: DistilBERT.* A smaller, faster, cheaper, and lighter version of BERT.
- **Ensemble Model:** We construct a soft voting ensemble combining the predictions of our best Bag-of-Words model, RoBERTa, and DistilBERT to leverage the strengths of both statistical and contextual representations. We are curious to see if combining the models leads to performance improvement.

# 4 Analysis

## 4.1 Establishing Baselines

The naive baseline predicts the most frequent class (non-disaster) for all instances. This serves as a lower bound for performance, ensuring that our complex models provide meaningful predictive value beyond simple probability guessing. Given the class imbalance (approx. 57% non-disaster vs 43% disaster), a model must achieve accuracy significantly higher than 57% to be considered useful. As seen, although the accuracy score is 57%, this baseline model achieves a score of 0 for F1 score, precision, and recall. This is because the baseline model only predicts the negative class (not a disaster), so true positive = 0 and precision and recall are 0. This is obviously not ideal for this application because we want to be cautious, identifying all potential real announcements even at the cost of false alarms. The baseline model is doing the opposite.

## 4.2 Bag of Words - Basic and Tuned

To move towards better performance, we first start with the simplest model. A basic model that works well is preferred over a complicated model that works the same, since the model's complexity may hinder computation time, which is especially integral in our application. We consider a basic bag of words with 5000 tokens, 40K total parameters. As seen in Table 1, this model already achieves much better performance than baseline, with an accuracy score of 0.798 and precision, recall, and F1 scores around $0.7 - 0.8$. Motivated to improve this performance further, we conduct a grid search of hyperparameters like number of layers, number of units, and dropout. This space is limited to a relatively lightweight model; e.g., maximum number of layers is 2 and units is 32, as once again, we prioritize computation time. Dropout is introduced to reduce overfitting. After iterating over 72 models, we obtain an improved result seen in `BoW(T)` virtually for free.

## 4.3 Bringing in Transformers

With simpler models established, we turn to context-aware models. Although we have already achieved good performance, context is crucial in our application and it would be unreasonable to ignore it completely. We implement a Transformer encoder from scratch to capture the contextual dependencies in tweets. Unlike BoW, which ignores word order, the Transformer uses self-attention to weigh the importance of different words relative to each other. We use a custom `TokenAndPositionEmbedding` layer to preserve sequence order and a `TransformerEncoder` layer with Multi-Head Attention; there are 400k trainable parameters in total. As seen, the vanilla transformer model `Transf.` achieves poorer performance than `BoW(FT)` across the board despite the 10X scale up in parameter size, most likely because we have not optimized it fully.

**Table 1:** Performance Comparison Across Models

| Metric | Base | BoW | BoW(T) | BoW(FT) | Transf. | Tr.(FE) | RoBERTa | DistilBERT | Ensemble |
|---|---|---|---|---|---|---|---|---|---|
| F1 Score | 0.000 | 0.757 | 0.769 | 0.771 | 0.704 | 0.680 | 0.812 | 0.801 | **0.826** |
| Accuracy | 0.570 | 0.798 | 0.817 | 0.818 | 0.760 | 0.707 | 0.841 | 0.829 | **0.853** |
| Precision | 0.000 | 0.783 | **0.842** | 0.839 | 0.749 | 0.640 | 0.825 | 0.802 | 0.839 |
| Recall | 0.000 | 0.733 | 0.707 | 0.713 | 0.664 | 0.725 | 0.798 | 0.800 | **0.815** |

## 4.4 Exploring Feature Engineering

Finding ways to enhance performance without increasing complexity, we turn to dataset enrichment. To incorporate our engineered features meaningfully, we analyze the correlation between our seven features and the target variable. The correlation matrix (visualized in the notebook) highlights which structural features, such as word count or the presence of URLs, are most indicative of real disaster tweets. For instance, real disaster tweets often contain more URLs (linking to news sources) and fewer mentions compared to

casual conversation. These features are integrated into the BoW and Transformer models to test if explicit structural information complements the semantic information learned from text. As seen in both columns, some important metrics like Recall improve from 0.707 to 0.713 for Bag of Words, and 0.664 to 0.725 in the transformer. This is a big gain as recall represents what proportion of real announcements we are able to classify, a crucial metric we want to optimize for in this application.

### 4.5 Leveraging BERT: Fine-tuning

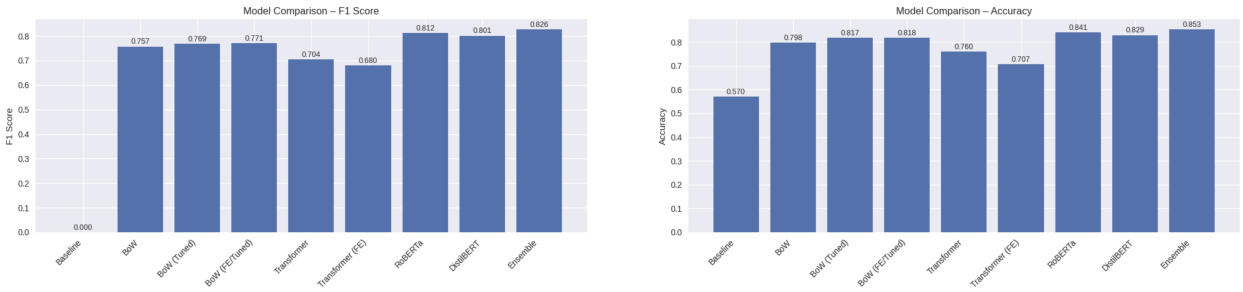
Additionally, we consider transfer learning using pre-trained models from the Hugging Face library: `roberta-base` and `distilbert-base-uncased`. These models have been pre-trained on massive corpora and possess deep linguistic knowledge. We fine-tune them on our disaster tweet dataset using a custom training loop with the `AdamW` optimizer and a linear learning rate scheduler with warmup. This approach allows the model to adapt its pre-learned representations to the specific nuances of disaster classification, typically yielding higher performance than training from scratch on a small dataset. As seen, `RoBERTa` and `DistilBERT` achieve performance in the $\approx 0.8$ across the board, an improvement from earlier texttBoW and transformer models. Interestingly, one pattern observed is that these models seem to have performance consistent across the metrics whereas earlier ones may perform a portion better at one metric compared to the other; this may be because they are leveraging global knowledge and thus have a more holistic understanding compared to earlier which may be skewed specifically to our dataset.

### 4.6 Synergy Through Ensemble

Having observed the strengthes and weaknesses of earlier mode, we decide to explore an ensemble model. We combine the probability outputs of `BoW(T)` model, `RoBERTa`, and `DistilBERT`. We apply a weighted soft voting strategy, assigning weights of 0.2, 0.4, and 0.4 respectively. This approach aims to mitigate the individual biases of each model, leveraging the robustness of pre-trained transformers while retaining the signal captured by the simpler `BoW` model. A happy result: this model performs the best across three metrics, achieving 0.826 F1 score, 0.853 accuracy, 0.839 precision (close second to 0.842 from `BoW (T)`), and 0.815 recall.

## 5 Evaluation

We evaluate all models using four metrics: F1-score, Accuracy, Precision, and Recall. The F1-score is our primary metric as it balances precision and recall, which is crucial for disaster detection where missing a real disaster (low recall) and flagging false alarms (low precision) are both detrimental.




(a) F1 Score Comparison

(b) Accuracy Comparison

**Figure 1:** Performance comparison of all the used models in both F1 score and Accuracy.

As seen in Figure 1a, the transformer models perform the worse, getting low 0.7 F1 score compared to BoW models and pre-trained models that achieve high 0.7 to 0.8. Although this is surprising as transformer captures context, it may be reasoned that our dataset is simply too small to make sense of these relationships captured in the context. This conclusion is reaffirmed once we consider the pre-trained models like `RoBERTa` and `DistilBERT` which is the same as the transformer just a larger architecture and a much larger dataset; their performance is much better. Figure 1b tells a similar story, where the tuned BoW and pre-trained models achieve strong performance. However, the Ensemble outperforms both, demonstrating that combining the strengths of BoW, which captures high-level frequency-based patterns and simple structural features, with pre-trained models, which encode deep contextual and semantic information, allows the models to complement each other and achieve superior overall performance.

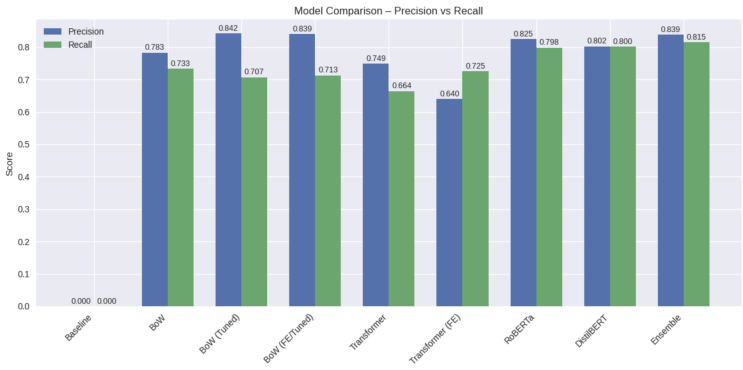



**Figure 2:** Precision vs. Recall Trade-off across all models

## 5.1 Takeaway

In this project, we successfully develop a pipeline to classify disaster tweets, progressing from simple baselines to state-of-the-art transformer models. Our analysis shows that while feature engineering adds value to simpler architectures like Bag-of-Words, its impact diminishes with more complex models like Transformers, which can implicitly learn these features. Pre-trained models like RoBERTa and DistilBERT provide the most significant performance boost, demonstrating the value of transfer learning in NLP tasks with limited labeled data. The overall best performing model is a synergy of both however, with the Ensemble model showing how creatively combining strengths can yield observable benefits.

## 5.2 Extension

Given that our work demonstrates the power of synergizing models, future research could explore advanced ensemble techniques, such as stacking or boosting, rather than simple weighted voting. We could also leverage domain-specific pre-trained models, such as BERTweet, to better handle the informal language and noise of Twitter. Further hyperparameter tuning of BERT models and ensemble weights may yield additional gains. Overall, this study shows that by effectively combining complementary modeling approaches, we can accurately classify short, noisy, and highly variable social media texts, which highlights techniques that are broadly applicable for extracting meaningful insights from short real-time data streams, one of the most common forms of communication today.